Generated: 2026-08-09 17:25

TheAnors - System Architecture & Technical Specification

Version: 1.0

Date: August 9, 2026

Architecture: Modular Monolith (Next.js)

---

1. System Overview

TheAnors is built as a modular monolith using Next.js. Single codebase, separate modules per workflow, shared infrastructure.

---

2. Architecture Layers

2.1 Frontend Layer

Location: `/app` (Next.js App Router)

Design System:

• Font: Arial (via Tailwind + custom CSS)

• Colors: #000000 (black), #FFFFFF (white), #DC143C (crimson red)

• Spacing: Tight (4px base unit, compact margins)

• Responsive: Mobile-first (viewport width 380px minimum)

Key Components:

1. **EngagementForm.tsx**

- Batch input for LinkedIn links

- Shows progress (8/30)

- Displays 3 comments per link

- Select + copy functionality

2. **CommentOptions.tsx**

- Shows 3 options for each post

- Display: option number, comment text, original link

- User can click checkbox "posted"

3. **CaptionDisplay.tsx**
- Shows 5 captions (LinkedIn, TikTok, IG, YouTube title, YouTube desc)
- Editable fields
- Copy buttons for each

4. **NewsletterBuilder.tsx**
- Theme input field
- Post link validator
- Generated content preview (Word export ready)

5. **ScriptingBuilder.tsx**
- Input: guidelines + topic/link
- Output: 3 script options
- Expandable details per option

2.2 API Layer (Backend)

Location: `/app/api` (Next.js API Routes)

Each API route handles:
• Request validation
• LLM API call (via abstracted client)
• Database writes (Supabase for live, Neon for analytics)
• Response formatting
• Error handling

2.3 Abstraction Layer

Location: `/lib/api-abstraction.ts`

This is the key to swappable providers.

Usage in API routes:

Swap provider = change `.env` variable. Done.

---

3. Database Architecture

3.1 Supabase (Real-time, Live Work)

Schema:

Supabase Features Used:
• Real-time subscriptions (live updates as user works)
• Row-level security (user can only see their data)
• Automatic backups
• Migrations support (for future schema updates)

3.2 Neon (Analytics & Training Data)

Schema: (Mirror of Supabase with additional tracking)

Neon Features Used:
• Async analytics queries (don't slow down live work)
• Separate connection pool
• Historical data for trend analysis
• Support for future model fine-tuning

3.3 Data Sync Strategy

Write Flow:

Why Dual DB:
• **Supabase:** Fast, real-time, for daily work
• **Neon:** Analytical queries don't slow live work

Sync Code:

3.4 Backup & Migration Strategy

Backups:

Migration Path (Future):

---

4. API Integration

4.1 Gemini LLM API

Endpoint: `https://generativelanguage.googleapis.com/v1/models/gemini-pro:generateContent`

Free Tier Limits:
• 1,000 requests per day
• 10 requests per minute

• 4 requests per minute for vision tasks

Usage in TheAnors:

Cost: Free (until ■500/month after free tier exhausted)

4.2 Google Vision API

For: OCR on Instagram carousel posts, TikTok captions

Endpoint: `https://vision.googleapis.com/v1/images:annotate`

Free Tier: 1,000 requests per month

Usage:

Cost: Free for MVP

4.3 Transcription APIs

Current Setup (from your existing tool):
• Groq (Whisper)
• Deepgram
• Gladia
• Assembly AI

Usage:

Cost: Use your existing API credits/tiers

4.4 Google Sheets API

For: Reading theme history spreadsheet

Endpoint: `https://sheets.googleapis.com/v4/spreadsheets/{spreadsheetId}/values/{range}`

Usage:

Cost: Free (limited to 100 requests/min)

---

5. Modular Structure

5.1 Module Organization

5.2 Module Pattern

Each workflow module follows this pattern:

Benefits:
• Each module is independent
• Easy to test
• Easy to add new workflows (just create new module)
• Shared utilities prevent duplication

---

6. File Export System

Location: `/lib/shared/export.ts`

---

7. Error Handling & Logging

Location: `/lib/shared/logger.ts`

Try-catch in all API routes:

---

8. Authentication

For MVP: Simple email + magic link (Supabase Auth)

Usage:

---

9. Environment Variables

---

10. Build & Deployment

10.1 Local Development

10.2 Build

10.3 Deployment (Vercel)

Cost: Free tier covers MVP traffic

10.4 Self-Hosted (Future VPS)

---

11. Testing Strategy

Unit Tests:

Integration Tests:

Testing Tools:
• Jest (unit + integration)
• Vitest (faster alternative)
• Playwright (end-to-end UI tests)

---

12. Performance Considerations

12.1 API Rate Limiting

Gemini (free tier): 1,000 requests/day, 10/min

Rate limiter:

12.2 Caching

For repeated prompts/configs:

Cache everything:
• Master prompts (24h TTL)
• User settings (1h TTL)
• Theme history (24h TTL)

12.3 Database Query Optimization

Supabase indexes:

Avoid N+1 queries:

---

13. Security Considerations

13.1 API Key Management

• **Never** commit `.env` to Git
• Use `.env.local` (local development) and Vercel's environment variables
• Rotate keys quarterly

13.2 Row-Level Security (RLS)

13.3 Input Validation

---

14. Monitoring & Observability

Basic setup:

Track:
• LLM API response times
• Database query times
• Error rates per workflow
• User engagement (which workflows used most)

---

15. Cost Breakdown (Monthly, Naira)

Service	Free Tier Limit	Cost (Naira)
Gemini API	1,000 requests/day	■0 initially, then ~■2,000
Google Vision API	1,000 requests/month	■0
Supabase	500 MB storage, 2GB bandwidth	■0 (or ■5,000 for Pro)
Neon	3 free databases, 3 GB storage	■0

| Vercel | 100 GB bandwidth | ■0 |

| **Total** | | **■0-7,000/month** |

Budget: Start on free tier. Move to paid tiers only when you exceed limits.

---

16. Deployment Checklist

• [ ] Set up GitHub repo with `.env.example`
• [ ] Configure Supabase project
• [ ] Configure Neon project
• [ ] Set up Gemini API key
• [ ] Configure Google Cloud Vision
• [ ] Create Vercel project
• [ ] Connect Vercel to GitHub
• [ ] Set environment variables in Vercel
• [ ] Run `npm install` + `npm run build` locally (test)
• [ ] Deploy to Vercel
• [ ] Test on production URL
• [ ] Set up Sentry for error tracking
• [ ] Configure Supabase backups
• [ ] Document setup in README

---

17. Glossary

Modular Monolith: Single codebase with separate modules per feature, deployed as one unit.

Abstraction Layer: Code that hides implementation details (e.g., swappable LLM providers).

RLS: Row-Level Security—database enforces per-row access control.

TTL: Time-to-Live—how long cached data stays valid.

Rate Limiting: Throttle API requests to stay within quotas.

Architecture Status: Ready for Build
Next Step: Design System Specification + Component Library